ELA: English Language Assistant

Recursive Linguistic Elements, Multimodal Processing, and Explainable Support for Authentic English Input

Vladyslav Rastvorov

R00274535

This thesis has been submitted in partial fulfillment for the degree of

Bachelor of Science (Honours) in Computer Systems

Faculty of Engineering and Science
Department of Computer Science
Munster Technological University Cork

Research Supervisor:	Dr. Alex Vakaloudis
Implementation Supervisor:	Dr. Nasir Ahmad
Academic Year:	2024–2025
Submission Date:	April 2026
Live system:	https://el-a.uk
Source code:	https://github.com/welsol21/FYP_LLM

Declaration of Authorship

This report, *ELA: English Language Assistant*, is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science (Honours) in Computer Systems at Munster Technological University Cork. I, Vladyslav Rastvorov, declare that the work presented in this thesis is substantially my own except where explicitly stated.

- This work was carried out wholly or mainly while in candidature for the above degree at Munster Technological University Cork.
- Where any part of this thesis has previously been submitted for another qualification, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given.
- I have acknowledged all main sources of help.
- Where the thesis is based on work carried out jointly with others, I have made clear what was done by others and what I contributed myself.

Signed: _____

Date: _____

Abstract

This thesis presents the continued development of the English Language Assistant (ELA), a full-stack language learning system designed to make authentic English input more accessible without simplifying or rewriting the source material. ELA ingests text, audio, and video, then transforms the input into a hierarchical contract of *Recursive Linguistic Elements* (RLE), where each sentence is represented as a structured tree of sentence, phrase, and word nodes enriched with grammatical, pedagogical, and bilingual metadata.

The implemented system combines deterministic linguistic processing and machine learning in a rules-first architecture. spaCy-based parsing, a deterministic skeleton builder, and a rule-based tense-aspect-mood layer provide the structural truth layer. Additional enrichments include CEFR estimation, pedagogical note generation, translation, phonetic transcription, vocabulary export, and runtime persistence. A React/Vite progressive web application and a runtime service expose these results through an interactive visualizer and media-oriented workflow.

Alongside the engineering contribution, the project examines a central modelling insight that emerged during note-generation experiments: performance improved substantially when raw lexical notation was replaced by structurally abstract placeholder templates. This observation is interpreted as a *context–pattern duality*: during training, diverse contextual examples carve stable internal patterns; during inference, frozen patterns impose structure on unseen context. In this thesis, that idea is treated as an empirical observation and explanatory framework rather than a formal theorem.

The final outcome is not only a prototype application but an implemented and partially validated architectural framework for explainable, AI-assisted language learning. At the same time, the thesis documents current limitations, including unresolved runtime and pipeline regressions, incomplete stabilization of some multimodal paths, and the gap between research claims and fully production-hardened behavior. The work therefore positions ELA as both a working system and an ongoing research programme.

Acknowledgements

I would like to thank my supervisors, Dr. Alex Vakaloudis and Dr. Nasir Ahmad, for their guidance across both the research and implementation dimensions of this project. I am also grateful to my family for their continuing support throughout the development of ELA, and to Munster Technological University Cork for providing the environment in which this work could take shape.

Contents

1 Introduction **1**

1.1 Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Research Questions . . . 1
1.4 Objectives . . . 2
1.5 Contributions . . . 2
1.6 Thesis Structure . . . 3

2 Background and Related Work **4**

2.1 Computer-Assisted Language Learning . . . 4
2.2 Authentic Materials and Cognitive Support . . . 4
2.3 NLP Parsing and Structured Analysis . . . 5
2.4 Tense, Aspect, and Mood . . . 5
2.5 CEFR Estimation . . . 5
2.6 Sequence-to-Sequence Models and T5 . . . 6
2.7 Automatic Speech Recognition and Translation . . . 6
2.8 Explainability as a Systems Property . . . 6
2.9 Gap Analysis . . . 7

3 Previous Research Foundation and Project Evolution **8**

3.1 Relationship to the Earlier Research Thesis . . . 8
3.2 From Conceptual Prototype to Implemented System . . . 8
3.3 Practical Evolution Through Repository Milestones . . . 9
3.4 Research Continuity . . . 11
3.5 What Is New in This Thesis . . . 11
3.6 Methodological Approach of This Thesis . . . 11
3.7 Project Risks . . . 12

4 The RLE Data Model **13**

4.1 Design Rationale . . . 13
4.2 Hierarchy . . . 13

4.3 Core Fields . . . 13
4.4 Validation Modes . . . 14
4.5 How the Contract Evolved in Practice . . . 14
4.6 Contract Example . . . 15
4.7 What the Canonical Sample Shows . . . 16
4.8 Why Hierarchical JSON . . . 16

5 Context–Pattern Duality and Placeholder Generalisation 18
5.1 Observed Problem . . . 18
5.2 Observed Improvement . . . 18
5.3 Empirical Path to the Claim . . . 18
5.4 Interpretive Framework . . . 19
5.5 Lexical Invariance . . . 19
5.6 Scope of the Claim . . . 20
5.7 Cross-Domain Evidence: The Same Pattern in Weather Forecasting . . . 20

6 System Architecture and Implementation 22
6.1 System Overview . . . 22
6.2 Technology Stack . . . 23
6.3 Backend Linguistic Pipeline . . . 23
6.4 Runtime Service and API Layer . . . 24
6.5 Frontend . . . 25
6.6 PWA and End-to-End Workflow Evidence . . . 26
6.7 Media Processing . . . 26
6.8 Persistence and Export . . . 27
6.9 Contract Safety as an Architectural Principle . . . 27
6.10 Deployment and Service Boundaries . . . 27

7 Dataset and Corpus Engineering 29
7.1 Why Dataset Engineering Matters . . . 29
7.2 Source Families . . . 29
7.3 Curation and Stabilization . . . 29
7.4 Deduplication and Quality Control . . . 30
7.5 Placeholder Construction . . . 30
7.6 Template Opportunity Analysis . . . 31
7.7 Measured Template Space . . . 31
7.8 From Free-Form T5 to Controlled Rendering . . . 32
7.9 Engineering Trade-off . . . 32
7.10 How the Dataset Strategy Evolved: From Raw Corpora to Grammar Books 33

8 Evaluation **37**
8.1 Evaluation Approach 37
8.2 Evaluation Method 37
8.3 What the Current Evidence Supports 38
8.4 Classifier Path and Controlled Generation Evidence 38
8.5 Concrete Tabular Evaluation Snapshot 38
8.6 Comparative Evidence and Intermediate Results 39
8.7 What the Current Evidence Does Not Fully Support 40
8.8 Representative Findings from Review 40
8.9 Overall Assessment 41
8.10 End-to-End Integration Perspective 41
8.11 Objectives-to-Evidence View 41

9 Discussion, Limitations, and Future Work **42**
9.1 Discussion 42
9.2 From LLM Pattern Learning to a General Scientific Method: A Pattern-Sequence Hypothesis 42
9.3 Limitations 47
9.4 Future Work 48

10 Conclusion **50**

A Repository Evidence Sources **51**

B Evaluation Artifact Index **53**

C Pipeline Template Examples **54**

Bibliography **58**

List of Figures

6.1 High-level architecture of the implemented ELA system 22

List of Tables

4.1 Representative field groups in the canonical sample contract 16
4.2 Hierarchical RLE versus flatter annotation approaches 17

6.1 Representative technology stack of the current ELA implementation . . . 23
6.2 Rules-first ownership in the implemented ELA pipeline 24
6.3 Representative runtime/API responsibilities described in the current project documentation 25

8.1 Selected values from the 2026-03-05 tabular joint-profile evaluation artifact 39
8.2 High-level objective-to-evidence mapping for the current thesis 41

9.1 Structural parallel between the ELA and weather forecasting pattern-sequence approaches 45
9.2 Current implementation limitations and their practical impact 48

A.1 Representative repository evidence sources used in this thesis 52

B.1 Evidence categories and their interpretive strength in the current thesis . 53

C.1 Elementary-level passthrough note examples from the training corpus . . 55
C.2 Intermediate-level slot-templated note examples from the training corpus 56

List of Abbreviations

ASR	Automatic Speech Recognition
CALL	Computer-Assisted Language Learning
CEFR	Common European Framework of Reference for Languages
FR	Functional Requirement
IPA	International Phonetic Alphabet
LLM	Large Language Model
ML	Machine Learning
NFR	Non-Functional Requirement
NLP	Natural Language Processing
POS	Part of Speech
PWA	Progressive Web Application
RLE	Recursive Linguistic Elements
TAM	Tense, Aspect, and Mood
WER	Word Error Rate

Chapter 1

Introduction

1.1 Motivation

Most digital language-learning systems lower difficulty by reducing or simplifying the source material. This makes early progress easier, but it also distances the learner from real English as it appears in speech, media, and written communication. Authentic input is richer, more motivating, and more representative of actual language use, but it is also harder to process without support. The central motivation of ELA is therefore not to simplify authentic English, but to make it explainable.

1.2 Problem Statement

The problem addressed in this project is the lack of a transparent, learner-facing system that can process authentic English across text, audio, and video, preserve the original linguistic structure, and expose just enough grammatical and bilingual support to improve comprehension. Existing systems typically optimize for fluency drills, vocabulary repetition, or black-box assistance. ELA instead targets explainable decomposition of authentic input into interpretable linguistic units.

In scope for this thesis are the implemented architecture, the linguistic contract, the multimodal runtime path, the frontend-facing visualization flow, and the experimental line around note-generation abstraction. Out of scope are large-scale classroom deployment and strong claims of measured learning gain through user studies. This boundary is intentional: the thesis evaluates a research-engineering system, not a fully validated educational product.

1.3 Research Questions

This thesis is guided by three research questions:

1. How can a rules-first linguistic pipeline produce structured, pedagogically useful representations from authentic media without simplifying the original input?
2. What representation model best captures sentence, phrase, and word relationships while remaining stable enough for validation, persistence, and visualization?
3. To what extent can structural abstraction, particularly placeholder-based modelling, improve the generalization of generated pedagogical notes?

1.4 Objectives

The implementation phase of ELA is organized around the following objectives:

1. Build a multimodal processing pipeline that ingests text, audio, and video and converts them into structured sentence-level representations.
2. Formalize a hierarchical contract that can carry grammatical, pedagogical, and bilingual metadata in a stable and validated form.
3. Deliver a learner-facing application that exposes linguistic structure through an interactive visualizer, export flow, and media-oriented workflow.
4. Investigate whether abstract structural templates support stronger note-generation generalization than raw lexical notation.

1.5 Contributions

The main contributions of this thesis are:

- an evolved implementation of ELA as a full-stack system rather than a concept-level prototype, deployed at `https://el-a.uk` and available as open-source at [16];
- the formalization and operationalization of the RLE contract as the core data interface between backend, storage, and frontend;
- a rules-first linguistic pipeline integrating parsing, phrase construction, TAM enrichment, CEFR estimation, translation, phonetic support, and pedagogical notes;
- a research interpretation of the placeholder strategy for note generation through the context–pattern duality;
- a critical evaluation of the current system, including implemented capabilities, empirical evidence, and unresolved limitations.

Taken together, these contributions position the thesis as both an implementation report and an argument about representational design. The software contribution is

the operational ELA stack itself. The research contribution is the claim that stronger pedagogical behaviour emerged when linguistic variation was reorganized into more stable structural forms.

1.6 Thesis Structure

The remainder of this report is structured as follows. Chapter 2 reviews the background fields relevant to ELA. Chapter 3 positions the current work as a continuation of the earlier research thesis and explains the project evolution. Chapter 4 defines the RLE data model. Chapter 5 examines the context–pattern duality. Chapter 6 presents the implemented system architecture. Chapter 7 discusses dataset and corpus engineering. Chapter 8 evaluates the system. Chapter 9 discusses limitations and future work. Chapter 10 concludes the thesis.

Chapter 2

Background and Related Work

2.1 Computer-Assisted Language Learning

Computer-Assisted Language Learning (CALL) provides the pedagogical backdrop of ELA. Classical CALL systems focused on drills, exercises, and tightly constrained curricula [4], while later work emphasized broader software-mediated support for second-language acquisition [5]. More recent systems incorporate adaptation, learner modelling, multimodal content, and mobile delivery. Even so, a large portion of practical CALL still prioritizes task completion and fluency support over structural transparency. The learner receives an answer, a translation, or a corrected sentence, but not necessarily an interpretable representation of why the form is what it is.

ELA aligns with a narrower strand of CALL that values learner autonomy, authentic materials, and graduated support. In that tradition, software should not simply replace the learner's analysis; it should externalize part of the analysis in a form that the learner can inspect. This matters especially in independent learning scenarios, where the system must function not only as an exercise engine but as a structured explanatory companion.

2.2 Authentic Materials and Cognitive Support

Authentic materials are attractive because they preserve the lexical, syntactic, pragmatic, and stylistic richness of real communication. This emphasis is well established in language-teaching literature on authenticity and materials design [1]–[3]. They expose the learner to naturally occurring tense shifts, ellipsis, collocation, discourse markers, and speaker intent in a way that sanitized textbook examples often do not. At the same time, the very properties that make authentic input valuable also make it cognitively demanding.

This creates a design tension. If the material is simplified too aggressively, authenticity is lost. If it is left untouched without support, the learner can become overloaded. ELA

addresses this gap by layering support over the original input rather than rewriting it. The project therefore sits at the intersection of authenticity, scaffolding, and transparency: it tries to preserve the original utterance while exposing interpretable structure around it.

2.3 NLP Parsing and Structured Analysis

The system relies on modern NLP tooling to obtain syntactic and morphological signals, especially tokenization, dependency parsing, lemmatization, and part-of-speech tagging. These signals are useful, but they are not themselves a learner-facing representation. Raw parser output is often unstable, implementation-specific, and difficult to expose directly in a pedagogical interface.

ELA therefore treats parsing as an evidence source rather than as the final user model. Parser outputs are normalized into a controlled hierarchical representation that can support enrichment, validation, storage, and visualization. This distinction is important: the project is not merely a wrapper over an NLP library, but an attempt to convert low-level linguistic signals into a stable pedagogical contract.

2.4 Tense, Aspect, and Mood

Verb morphology is central to pedagogical explanation in ELA because tense, aspect, and mood are among the most common sources of learner confusion. In many learning tools, TAM interpretation is either hidden inside a black-box model or reduced to surface labels that are difficult to audit. ELA adopts a more conservative approach. The implemented system uses a deterministic rules layer over parser output to infer TAM-related information and to preserve traceable links between grammatical evidence and pedagogical phrasing.

This is not only an engineering convenience. It is also a methodological choice. A deterministic TAM layer makes failure analysis easier, allows targeted tests, and prevents downstream note generation from becoming the sole source of grammatical truth.

2.5 CEFR Estimation

The Common European Framework of Reference for Languages offers a widely recognized scale for expressing learner difficulty and progression. In ELA, CEFR is not treated as a complete pedagogical theory, but as a practical enrichment layer that helps characterize sentence and node difficulty in a form familiar to teachers and learners.

The repository shows that multiple approaches were explored, including classifier-driven and tabular strategies. The implemented system converges toward a lighter-weight

deployment path because it is easier to integrate into a contract-oriented architecture and easier to reason about operationally. This reflects a broader pattern in the project: simpler, auditable components are often preferred when they fit the explanatory goals better than heavier black-box alternatives.

2.6 Sequence-to-Sequence Models and T5

Sequence-to-sequence models are relevant because pedagogical note generation in ELA is not a pure classification problem. It requires controlled wording generation from structured inputs. T5-style models are a natural fit for this kind of mapping, especially when the input can be expressed as a text-like notation carrying grammatical state [7].

However, the project evidence suggests that representation strategy matters at least as much as model choice. The strongest note-generation behavior did not come simply from scaling model size or training longer. Instead, it emerged when the training signal was restructured through placeholders and canonicalized patterns. This observation motivates the later discussion of context–pattern duality.

2.7 Automatic Speech Recognition and Translation

Authentic language learning is not restricted to text. Spoken input introduces pronunciation, pacing, disfluency, and subtitle alignment problems that do not appear in static text. ELA therefore extends to audio and video through ASR, sentence segmentation, subtitle generation, and translation enrichment.

Whisper-based transcription and M2M100-based translation represent the local model path visible in the codebase and documentation, while additional providers remain optional or comparative extensions [8], [9]. This multimodal dimension makes ELA more ambitious than a text-only parser, but it also increases integration fragility because sentence boundaries, timestamps, and translated artifacts must remain synchronized across several layers.

2.8 Explainability as a Systems Property

An important implication of the preceding literature is that explainability in language-learning software cannot be reduced to a single model output. A system becomes explainable when its internal representation, validation boundaries, interface structure, and learner-facing presentation all support inspection. This is why ELA invests so heavily in an intermediate contract. The RLE structure is not just a storage convenience; it

is the mechanism through which parser evidence, grammatical interpretation, bilingual support, and frontend visualization remain aligned.

2.9 Gap Analysis

The main gap targeted by ELA is the absence of a single learner-facing platform that takes authentic multimodal English input and converts it into a structured, hierarchical, explainable, and exportable pedagogical representation. Existing tools may offer translation, subtitles, grammar help, or media playback, but ELA's distinguishing aim is the combination of:

- authentic media ingestion;
- hierarchical structural representation;
- explicit validation of the intermediate contract;
- learner-facing visualization and export;
- a research path toward abstraction-based pedagogical generation.

Chapter 3

Previous Research Foundation and Project Evolution

3.1 Relationship to the Earlier Research Thesis

This thesis is not the first formal document produced around ELA. A previous research thesis established the conceptual foundations of the project: the rationale for explainability over simplification, the importance of authentic input, the Recursive Linguistic Elements idea, and the pedagogical role of an interactive visualizer. That earlier work was primarily concerned with framing the problem, defining the object model, and demonstrating a prototype direction.

The current thesis continues from that base. Its focus is no longer the justification of ELA as an idea, but the implementation, validation, and critical assessment of ELA as an engineering system. This distinction matters: the current contribution should be evaluated on what was operationalized, tested, stabilized, and discovered during implementation.

3.2 From Conceptual Prototype to Implemented System

The project evolved along three main axes:

1. **Architectural maturation.** Earlier design descriptions emphasized a concept-driven prototype and a more speculative integration of external AI services. The current implementation shifts the core truth layer toward deterministic parsing, rules, and validation, with generated content treated as an enrichment path rather than the foundation of correctness.

2. **Expansion of the data contract.** The RLE concept was converted into a more

formal and stricter contract that now includes node identifiers, parent relationships, source spans, dependency information, trace metadata, translations, and multiple enrichments.

3. **Operational workflow.** The project now includes a runtime service, client persistence, media pipeline, frontend routes, export paths, and a large test surface. This moves ELA closer to a usable software product while also exposing implementation weaknesses that a purely conceptual report could avoid.

3.3 Practical Evolution Through Repository Milestones

The repository artifacts make this evolution unusually visible because design decisions were documented and then implemented in clearly traceable stages. The earliest implementation framing emphasized a deterministic foundation first. The implementation proposal for the linguistic notes pipeline defined the initial practical gap very explicitly: before any serious model path could be trusted, the system needed a deterministic core consisting of parser-driven skeleton construction, TAM rules, strict validation, and a production-safe inference runner. In other words, the first major implementation move after the research thesis was not toward a larger model, but toward a safer and more reproducible structural baseline.

This foundation was then pushed into a stronger contract form. The dated repository artifacts show a sequence of changes that tightened the sentence contract, unified note context flow, stabilized grammar blueprints, and eventually migrated the system to a recursive node hierarchy. That progression matters for the thesis because it marks the point where RLE stopped being mainly a conceptual representation and became an operational interface shared by inference, validation, runtime payloads, and the frontend visualizer. The curator progress report later describes this stage as a stabilization of a contract-compliant, structurally cleaner baseline with stricter validation and wider test coverage.

The next major shift was architectural rather than purely linguistic: the project moved toward a client-first product model. The product specification dated 2026-02-17 defines this stage clearly. Heavy interactive work, local projects, media handling, and learner edits were pushed toward the client, while the backend was narrowed to minimal orchestration and sentence-contract support. This decision then appears in later implementation artifacts through local SQLite persistence, runtime capability flags, media routing policy, frontend-ready runtime media service, project-scoped upload and processing paths, and eventually a React frontend slice with visualizer and media workflow support. As a result, the project moved from “pipeline plus prototype UI” toward a

coherent application architecture with explicit offline/online behavior.

At the same time, the modelling strategy continued to evolve. Repository artifacts from February and March 2026 show the emergence of a classifier-oriented pedagogy stack: curriculum specifications, classifier orchestration, blueprint traceability checks, tabular CEFR baselines, and full-ladder evaluation checkpoints. This phase did not eliminate generation, but it changed its role. By March 2026, the technical specification no longer described T5 as a component that could freely produce pedagogical structure. Instead, the target architecture became classifier-first and contract-safe: deterministic parsing and rules own structure, tabular ML owns CEFR, and T5 is confined to controlled lexical realization from a deterministic template.

The intermediate artifacts make this transition especially clear. A report from 2026-02-13 on the template-id experiment shows that the pipeline wiring was already deterministic, but that the dataset had collapsed to only ten active template IDs after balancing and still produced a practical fallback rate of 1.0 on the small regression suite. That is an important intermediate result: the project learned that structural control alone was not enough if the target space became too compressed. A later classifier execution checkpoint from 2026-02-25 shows the opposite kind of limitation. The orchestrator, quality gates, and controlled rewrite path were all operational, but the baseline freeze still used only four training rows and two development rows. In other words, by late February the architecture was becoming coherent faster than the data scale.

That transition is one of the clearest practical bridges from the earlier research thesis to the final implementation state. In the earlier thesis, the custom model path existed mainly as a promising future direction. In the implemented repository, that direction was tested, constrained, partially validated, and reorganized. The important result is not just "T5 was used." The more important result is that repeated practical work led to a narrower and more defensible design: blueprint preservation, field ownership, template expansion, controlled rendering, explicit post-render validation, and a willingness to treat weak intermediate results as design evidence rather than as something to hide.

The final major practical axis was multimodal and operational integration. The later repository artifacts include media upload and extraction paths, ASR-first local processing, translated media artifact generation, translation overlays, phonetic enrichment, synonym enrichment, Docker deployment notes, and PWA integration checks. These changes are not peripheral. They show how the original educational vision was extended from text analysis to a broader learner workflow involving files, projects, media, review, persistence, and export. They also explain why the current thesis must discuss regressions and integration mismatches: once the project became an end-to-end system, weaknesses at runtime boundaries became part of the real research record rather than hidden implementation detail.

3.4 Research Continuity

Despite the architectural shift, the underlying continuity is strong. Both the earlier thesis and the current project maintain the same central educational premise: authentic English should remain intact, and support should be attached through structured interpretation rather than through simplification. The implementation phase did not replace that vision; it forced it into a more disciplined computational form.

3.5 What Is New in This Thesis

The new contributions are best understood in contrast to the earlier report:

- the earlier thesis introduced ELA and RLE conceptually; this thesis documents a running full-stack implementation;
- the earlier thesis argued for explainability; this thesis shows how that explainability is represented in contract fields, validators, runtime APIs, and frontend views;
- the earlier thesis treated the custom model path as future work; this thesis can discuss concrete experimentation with T5 note generation, placeholders, and dataset engineering;
- the earlier thesis framed the system optimistically; this thesis must account for real regressions, trade-offs, and incomplete stabilization.

3.6 Methodological Approach of This Thesis

The work reported here follows a hybrid engineering-research methodology. It is not a purely theoretical study, nor is it a simple build diary. Instead, the thesis combines:

- architectural analysis of the implemented codebase;
- examination of project specifications, README-level documentation, and internal reports;
- review of test coverage and targeted failing tests in the current codebase;
- analysis of selected evaluation artifacts produced during model and pipeline development;
- comparison with the earlier research thesis in order to isolate what is genuinely new in the final-year implementation phase.

This methodological framing matters because ELA evolved over time. Different documents in the repository describe different moments of the project. The thesis therefore

treats the repository as a historical engineering record rather than as a single perfectly synchronized artifact.

3.7 Project Risks

From the perspective of the final thesis, the main risks are not only technical but evidential:

- **claim drift**: older project narratives may overstate capabilities that the current code does not fully stabilize;
- **metric drift**: different evaluation checkpoints may produce different headline numbers;
- **integration risk**: runtime, frontend, and pipeline layers evolve at different speeds, so stricter validation may expose mismatches;
- **scope risk**: the system spans too many subsystems for every path to be equally mature at submission time.

Recognizing these risks improves the quality of the report. It encourages careful wording, checkpoint-specific evidence, and explicit distinction between implemented baseline, experimental path, and future direction.

Chapter 4

The RLE Data Model

4.1 Design Rationale

Recursive Linguistic Elements (RLE) form the central data contract of ELA. The purpose of RLE is to translate parser output and downstream enrichments into a single representation that is stable enough for storage and validation, expressive enough for machine processing, and simple enough to be rendered in a learner-facing visualizer.

4.2 Hierarchy

The implemented hierarchy uses three conceptual layers:

1. **Sentence** as the root pedagogical and analytical unit;
2. **Phrase** as an intermediate structure carrying meaningful chunks such as noun phrases, prepositional phrases, or other grouped spans;
3. **Word** as the terminal lexical unit carrying finer grammatical metadata.

This structure is recursive because every non-terminal node contains a `linguistic_elements` array of child nodes. That makes the contract easy to traverse, validate, render, and export.

4.3 Core Fields

The contract uses a mixture of required and optional fields. The exact field set evolved during implementation, but the most important fields include:

- `type`, `content`, and `part_of_speech`;
- `tense`, `aspect`, `mood`, `voice`, and `finiteness`;
- `node_id`, `parent_id`, and `source_span`;

- `grammatical_role`, `dep_label`, and `head_id`;
- `linguistic_notes`, `notes`, and note provenance-related fields;
- `translations`, `phonetic`, `synonyms`, and `cefr_level`;
- `schema_version` and quality-control trace fields.

4.4 Validation Modes

The implementation retains two validation modes:

- **v1**, intended as the backward-compatible mode;
- **v2_strict**, intended as the current production-facing mode with stronger field requirements.

This dual-mode design is useful in theory because it enables controlled migration. In practice, the implementation review shows that validation compatibility is still imperfect, especially around null handling in TAM fields and around runtime payload expectations.

4.5 How the Contract Evolved in Practice

The RLE contract did not appear fully formed. It was expanded through a series of practical corrections driven by downstream needs. Early contract work focused on making the pipeline valid and testable at all. Later implementation stages added normalized morphology features, dependency linkage fields, grammatical roles, node metadata, trace fields, strict validation mode, modal-perfect policy enforcement, translation validation, and subtree-level backoff diagnostics.

This evolution matters because each addition corresponds to a concrete pressure from the rest of the system. Node identifiers and parent links were needed for deterministic graph identity and frontend selection. Source spans were needed for alignment, translation projection, and edit targeting. Grammatical roles and dependency fields were needed so that the tree was not merely a nested display object, but a linguistically meaningful structure. Trace fields and rejected-candidate statistics were needed because once the project started filtering, retrying, and validating generated notes, the contract also had to record why certain outputs were accepted or rejected.

The migration to recursive hierarchy is especially important in this story. The dated repository artifacts show an explicit transition away from legacy flat rules toward the recursive node contract. That move made the visualizer, runtime payloads, validator logic, and later controlled note-generation path all speak the same structural language. It also made the contract more demanding: once recursive identity and field ownership

became central, schema drift and null-handling issues became more visible rather than easier to ignore.

4.6 Contract Example

Listing 4.1 presents a simplified example of a sentence-level RLE node.

Listing 4.1: Simplified RLE contract example

```
{
  "type": "Sentence",
  "content": "She should have trusted her instincts.",
  "schema_version": "v2",
  "node_id": "s1",
  "parent_id": null,
  "source_span": { "start": 0, "end": 39 },
  "part_of_speech": "sentence",
  "grammatical_role": "root",
  "tense": null,
  "aspect": "perfect",
  "mood": "modal",
  "voice": "active",
  "finiteness": "finite",
  "translations": {
    "m2m100": {
      "source_lang": "en",
      "target_lang": "ru",
      "text": "..."
    }
  },
  "linguistic_notes": {
    "elementary": "...",
    "intermediate": "...",
    "advanced": "..."
  },
  "linguistic_elements": []
}
```

4.7 What the Canonical Sample Shows

The repository's canonical sample contract makes several implementation decisions explicit. A sentence node is not just a container for text. It is a convergence point for structural, pedagogical, bilingual, and traceability layers:

- sentence and phrase nodes can carry TAM values and pedagogical notes;
- node-level translations and phonetic values are attached as first-class fields rather than external lookups;
- `grammar_classes`, `note_blueprints`, and `note_generator_version` preserve the distinction between structural evidence and rendered learner-facing wording;
- `source_span`, `node_id`, and `parent_id` are intended to make every node referentially stable enough for UI selection, validation, and later editing.

The field-ownership matrix dated 2026-03-04 clarifies why this sample matters architecturally. The contract is not meant to be a place where one model predicts everything. Instead, different field families belong to different sources of truth: parser and builder rules own structure, rules-first logic owns TAM and grammatical interpretation, tabular ML owns CEFR, and T5 is constrained to learner-facing note wording only. This is one of the most important implementation outcomes of the whole project, because it turns the contract from a passive schema into an active boundary between deterministic truth, predictive enrichment, and generated phrasing.

Table 4.1: Representative field groups in the canonical sample contract

Field group	Function in the system
Core identity	`type`, `content`, `node_id`, `parent_id`
Structural alignment	`source_span`, `grammatical_role`, `dep_label`, `head_id`
Grammar interpretation	`tense`, `aspect`, `mood`, `voice`, `finiteness`, `tam_construction`
Pedagogical layer	`linguistic_notes`, `grammar_classes`, `note_blueprints`
Learner support	`translations`, `phonetic`, `synonyms`, `cefr_level`
Traceability	`schema_version`, `note_generator_version` and related quality fields

4.8 Why Hierarchical JSON

Hierarchical JSON was chosen over flatter annotation forms for several reasons:

- it matches the recursive structure required by the visualizer;
- it allows sentence, phrase, and word nodes to share a common schema family;
- it supports machine validation and human inspection simultaneously;
- it simplifies serialization for runtime storage, export, and testing.

Table 4.2: Hierarchical RLE versus flatter annotation approaches

Criterion	Hierarchical RLE	Flat annotation
Structural nesting	Native	Requires external linking
Frontend rendering	Direct tree traversal	Extra assembly layer needed
Validation	Node-by-node plus subtree rules	Easier locally, weaker globally
Pedagogical readability	Strong for progressive disclosure	Often fragmented
Export simplicity	High for JSON workflows	Higher for spreadsheets only

Chapter 5

Context–Pattern Duality and Placeholder Generalisation

5.1 Observed Problem

During the note-generation experiments, a recurring issue appeared: when the training material preserved too much lexical surface detail, the model tended to memorize examples instead of learning transferable pedagogical structure. This was especially problematic because ELA requires generalization to previously unseen vocabulary inside familiar grammatical patterns.

5.2 Observed Improvement

Performance improved after restructuring the target space around more abstract templates and placeholders. Instead of tying the generation task tightly to concrete lexical tokens, the revised setup emphasized structural roles and reusable pattern slots. The practical result was a stronger ability to produce pedagogical notes for unseen examples that shared grammatical structure with the training material.

5.3 Empirical Path to the Claim

This idea did not appear as a finished theory from the start. It emerged through several practical turns. An early template-ID experiment on 2026-02-13 confirmed that deterministic template wiring was in place, but it also exposed a serious problem: after balancing, the dataset had only 511 rows, only 10 unique targets, and a duplicate ratio above 0.98, while both the refreshed baseline and the template-ID model produced a fallback rate of 1.0 on the small regression probe. In other words, the first attempt to constrain the target space became too collapsed to be useful.

This failure is important because it shows what the project actually learned. Placeholder-based abstraction helped only when it expanded transferable structure without collapsing target diversity. The later strategy documents therefore move away from "T5 predicts the pedagogical truth" and toward a classifier-first architecture in which compact structural prompts predict a closed template space and the final note is rendered from slots. This is a narrower but much more defensible use of abstraction.

The later artifacts also show why the interpretation remained valuable after that failed intermediate stage. By March 2026, the pipeline documentation and contract field-ownership rules had converged on the same separation: the contract and blueprints carry structural truth, while T5 is allowed to realize wording only. A phase-1 execution report dated 2026-02-25 confirms that this controlled rewrite path preserved classifier-owned blueprint fields while still producing user-facing generated notes. That is a much stronger foundation for the current thesis claim than raw seq2seq optimism would have been.

5.4 Interpretive Framework

This thesis interprets that behavior through a context–pattern duality:

- during training, many contextual examples shape the internal pattern space of the model;
- during inference, the now-frozen patterns interpret new contextual instances.

In simple terms, context forms pattern in the open training phase, while pattern forms context in the closed inference phase. This is not presented here as a formal theorem. It is an empirical observation arising from the project's modelling path and used as an explanatory framework for why placeholder abstraction improved generalization.

5.5 Lexical Invariance

The key idea behind the placeholder strategy is lexical invariance. If the model learns to associate structural markers with pedagogical interpretation, then the same pattern can be reused when the surrounding words change. This is especially valuable in language learning, where the educational meaning of a structure often outlives any single example sentence.

The measured artifacts support a bounded version of this claim. Sample inference reports from the later hybrid book-enriched note path show exact sentence-level matches for several structurally distinct cases, including second conditionals, do-support negation, passive-plus-perfect constructions, existential agreement, and yes/no questions with auxiliary support. At the same time, phrase-level examples in the same artifact still expose

unresolved slot-surface issues, because placeholder tokens such as `PART_OF_SPEECH` and `GRAMMATICAL_ROLE` were emitted too literally. This matters for the thesis argument: the project did not discover perfect abstraction, but it did discover that lexical invariance becomes useful when the right information is abstracted and the remaining wording is tightly controlled.

5.6 Scope of the Claim

The claim should remain bounded. This thesis does not prove a universal law of learning systems, nor does it attempt to derive a physics result from model training behavior. The contribution is narrower and more defensible:

- the project observed a generalization gain after switching from raw lexical notation to structurally abstract templates;
- the context–pattern duality provides a coherent explanation of that gain;
- the interpretation is useful for designing pedagogical generation tasks in ELA and similar systems.

5.7 Cross-Domain Evidence: The Same Pattern in Weather Forecasting

The scope statement above deliberately restricts the context–pattern claim to ELA and pedagogical note generation. However, the same structural observation emerged independently in a parallel domain during the same research period, which provides a degree of cross-domain confirmation that strengthens the interpretive framework without formally extending its scope.

In a companion forecasting project [15] using six years of hourly meteorological observations from a single Irish station, raw sensor time series for temperature, pressure, and wind speed were not fed directly to a sequence model. Instead, measurements were first compressed into sequences of analytical segments, each described by one of a small vocabulary of ODE-derived equation types — `constant`, `linear`, `exponential`, `harmonic`, and `damped_harmonic` for temperature, and a three-type vocabulary for pressure and wind speed — together with its fitted parameters and duration. Over the 2020–2026 training period this segmentation produces 10,055 joint segments with a mean duration of 5.3 hours. A 3-layer transformer encoder (d=128, 4 heads) was then trained to predict the next segment descriptor from a symmetric window of past ones. The model learns to predict the *type and parameters of the next analytical regime*, not individual numeric

measurements. On a 7-day evaluation period in February 2026, which included a 22 hPa frontal pressure drop, the ensemble achieved the following key results [15]:

- **Pressure MAE at h 1–12: 0.89 hPa**, a 34% improvement over naive persistence (1.35 hPa);
- **Temperature MAE at h 1–24: 1.42 °C**, matching persistence (1.47 °C) with no atmospheric physics;
- **Wind speed MAE overall: 5.62 kt**, beating persistence (7.21 kt) by 22%.

The structural parallel to ELA is direct. In ELA, placeholder tokens replace specific lexical items so that the training signal emphasizes structural grammatical relationships rather than surface word sequences. In the weather project, ODE equation-type labels replace raw sensor values so that the model learns structural meteorological regime transitions rather than noisy point measurements. In both cases, the practical benefit arises from the same representational move: elevating the learning target from surface instances to analytical structural units, and then training a sequence model on the grammar of those units rather than on the raw signal.

This convergence across two domains with no shared vocabulary, no shared data, and no shared model architecture is what gives the context–pattern duality its broader interest as an interpretive framework. The ELA case shows that structural abstraction improves pedagogical generalization in language learning. The weather case shows that analytical-unit decomposition supports regime-level forecasting in physical time series. Neither result was planned as a controlled test of the other, which makes the convergence more informative, not less. The pattern-sequence approach appears to be a property of the representational strategy, not an artefact of a particular domain or dataset.

Chapter 6

System Architecture and Implementation

6.1 System Overview

The implemented ELA system spans backend processing, runtime orchestration, storage, media handling, and a learner-facing frontend. At a high level, the architecture can be read as a pipeline from authentic input to explainable output.

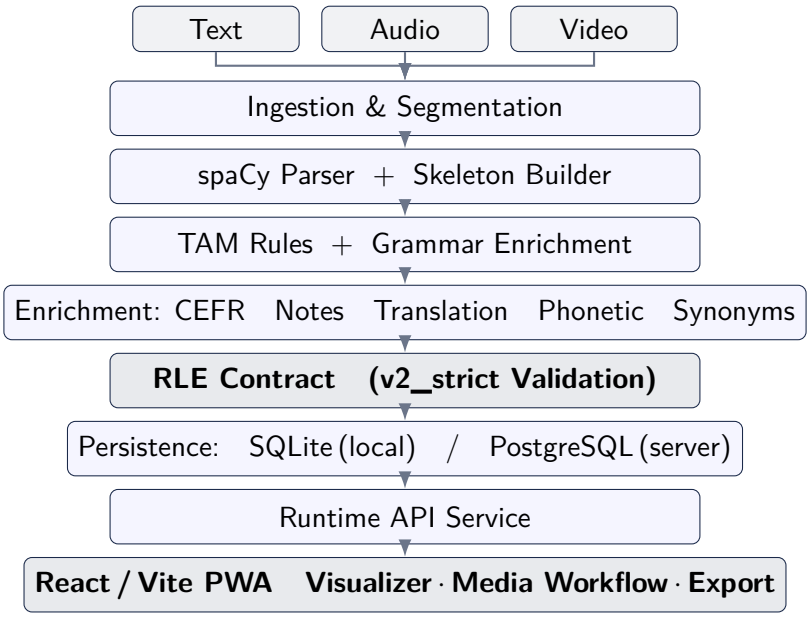

Figure 6.1: High-level architecture of the implemented ELA system

6.2 Technology Stack

Table 6.1: Representative technology stack of the current ELA implementation

Layer	Main technologies	Role in the system
Frontend	React, Vite, TypeScript, PWA packaging	learner-facing workflow, visualizer, file/project interaction
Desktop packaging path	Tauri	desktop-oriented delivery path alongside the web application
Runtime/API	Python runtime service, HTTP bridge	sentence-contract processing, media submission, job orchestration
Linguistic core	spaCy, deterministic skeleton builder, TAM rules	structural truth layer
Generated enrichment	controlled note-generation path, template transformation, optional T5-based rendering	user-facing pedagogical wording
Translation and speech	M2M100, Whisper, espeak-ng, ffmpeg-related media tools	translation, ASR, phonetics, subtitle/media processing
Storage	SQLite and PostgreSQL paths	local state, persisted analysis history, reproducibility
Deployment	Docker Compose, Nginx	service isolation and reproducible deployment boundaries

6.3 Backend Linguistic Pipeline

The core pipeline is implemented in Python under the `ela_pipeline` package. The project README and module structure indicate a rules-first design with the following major stages:

1. input ingestion and segmentation;
2. parser-backed skeleton construction;
3. deterministic TAM enrichment;
4. optional CEFR, note, translation, phonetic, and synonym enrichment;
5. contract validation;

6. persistence and export.

One of the most important architectural decisions is that structural truth is not left to a generative model. Parser output, rule logic, and validators form the trust boundary, while generated content is treated as controlled enrichment.

Table 6.2: Rules-first ownership in the implemented ELA pipeline

Layer	Main responsibility	Why this matters
spaCy + skeleton builder	structural graph, sentence/phrase/word hierarchy, POS, dependency-backed spans	keeps core representation deterministic and inspectable
TAM rules	tense, aspect, mood, voice, finiteness, TAM construction logic	prevents free-form generation from inventing grammar labels
Tabular or classifier path	CEFR-oriented predictive enrichment and blueprint support	separates classification from free-form wording
Controlled renderer / note path	lexical realization of learner-facing note text	constrains generation to wording rather than structure
Validator	contract safety, schema enforcement, field ownership boundaries	enforces architectural discipline across all later stages

6.4 Runtime Service and API Layer

Beyond offline inference scripts, the current system includes a dedicated runtime layer responsible for project management, client storage, media submission, background processing, UI state, and an HTTP bridge for the frontend. This is a major step beyond the earlier thesis, which focused more on conceptual architecture than on operational service boundaries.

The runtime layer also reveals one of the current weaknesses of the project: the service has become strict enough to expose inconsistencies between expected frontend payloads and actual sentence-node shapes. That makes the runtime layer useful both as an application service and as a form of architectural pressure test.

The runtime API is also concrete enough to describe as a product boundary rather than as an internal helper. The deployment documentation lists endpoints for project creation, selected-project state, file registration, media submission, backend-job status, visualizer payload retrieval, and edit application. This matters for the thesis because it

shows that the implementation phase did not stop at "produce JSON." It built a service layer through which the learner workflow could actually operate.

Table 6.3: Representative runtime/API responsibilities described in the current project documentation

Runtime concern	Role
Project and file state	manage learner projects, selected project state, and media registration
Sentence-contract path	expose text-to-contract processing through HTTP endpoints
Media submission	route uploads through local processing or policy-based rejection paths
Background job handling	support long-running tasks and status polling for media workflows
Visualizer payload delivery	provide contract trees back to the frontend in UI-facing form
Edit and synchronization layer	preserve local edits, retry paths, and resume-oriented workflow state

6.5 Frontend

The frontend is implemented as a React/Vite progressive web application. It organizes the user experience around projects, files, analysis, vocabulary, configuration, and a linguistic visualizer. Compared with the earlier Kivy-centred prototype framing, this frontend reflects a move toward a more modern web-oriented delivery model and a clearer separation between presentation and processing.

The frontend development note is particularly useful because it records the practical scope of what was actually implemented. It confirms a project-first flow of 'Projects -> Files -> Analyze', project-scoped file registration, stage progress reporting, artifact download actions, sentence-by-sentence visualizer navigation, translation-provider handling, quick node edit controls, and vocabulary export. It also records an important architectural subtraction: backend media-job UI controls were removed after the client-first runtime decision. That kind of removal is as important as an added feature because it shows that the final architecture was shaped by narrowing responsibilities, not only by accumulating them.

6.6 PWA and End-to-End Workflow Evidence

The repository also contains a dedicated PWA integration-test guide intended for pre-deployment verification. That document is useful because it describes the system not as isolated modules but as an end-to-end learner workflow:

1. open the application;
2. register service-worker readiness;
3. create a project;
4. upload a real media file;
5. open the analysis screen;
6. run the full pipeline from transcription through rendering;
7. verify the appearance of generated downloadable artifacts.

This is important evidence for the thesis because it shows that the project explicitly tests real browser behavior and long-running media paths, not only narrow unit-level logic. The same test guide also captures several resolved operational problems, such as model-installation readiness, Vite proxy configuration, service-worker cache staleness, and the move away from FFmpeg drawtext rendering toward a Canvas-based video rendering path. It therefore functions as both a verification checklist and a compact record of implementation lessons learned under real end-to-end conditions.

6.7 Media Processing

The media path extends ELA beyond text. Audio and video flow through extraction, transcription, sentence preparation, optional translation, and artifact rendering. The runtime also includes subtitle generation and media export paths. This makes ELA a multimodal learning system rather than a text-only analysis tool.

The implementation record shows that this path changed materially during development. Earlier stages experimented with backend-oriented media jobs, but the client-first product specification and later implementation artifacts narrowed the architecture toward local processing for interactive media work, explicit duration and file-size gates, and clearer offline/online behavior. This is a significant architectural result because it demonstrates that multimodality was not simply bolted on; it forced the project to make explicit decisions about privacy, runtime capability, failure boundaries, and where heavy processing should live.

6.8 Persistence and Export

The project contains both local client-side persistence and backend-oriented repository layers, with SQLite and PostgreSQL paths represented in the codebase. Export formats include JSON, CSV, and subtitle artifacts. This persistence strategy supports reproducibility, resumability, and inspection outside the application.

The persistence story is also historically meaningful. Implementation artifacts and product notes show a move from lightweight local repositories toward a more explicit split between device-local workspace state and backend-backed shared or synchronized state. That split helps explain several later design choices: selected-project persistence in the frontend, local edit capture, resumable media workflows, and the minimal-identity policy documented for backend deployment.

6.9 Contract Safety as an Architectural Principle

The technical specification dated 2026-03-20 is explicit that the next-stage architecture should remain classifier-first and contract-safe. In that design, the linguistic contract is not free-form model output. It is the source context from which later rendering is derived. This leads to clear ownership boundaries: the generator may realize wording, but it must not overwrite structural fields such as node identifiers, parent links, source spans, dependency fields, grammar classes, or CEFR predictions. That separation is one of the strongest engineering ideas in the project because it ties explainability to field ownership rather than to UI rhetoric alone.

6.10 Deployment and Service Boundaries

The Docker deployment documentation shows that ELA is no longer just a local research script collection. The intended deployed form has three clear service boundaries:

- PostgreSQL for persistent storage;
- an application container for inference, runtime APIs, and migrations;
- a frontend container serving the React interface through Nginx.

The same documentation is also useful because it avoids overclaiming. It explicitly distinguishes `online` and `offline` runtime modes, notes that model downloads may be disabled at runtime, and states that some features are unavailable in stricter offline or distributed policies. It also makes deployment policy visible as part of the architecture itself through environment-level constraints for media duration, media size, phonetic availability, deployment mode, and runtime download permissions. This is exactly the

kind of nuance the final thesis should keep, because it turns “offline-first” and “privacy-first” from slogans into constrained operational modes with explicit trade-offs.

Chapter 7

Dataset and Corpus Engineering

7.1 Why Dataset Engineering Matters

The implemented system is not driven by a single end-to-end model. As a result, dataset engineering matters differently across subsystems. CEFR estimation, note generation, and future model directions each depend on curated and interpretable training material rather than on generic scraped corpora alone.

7.2 Source Families

The repository indicates several source families and dataset-building paths, including curated corpora, open linguistic resources, and derived educational examples. The codebase and documentation reference sources such as UD-style corpora, OANC, Gutenberg-derived material, MASC, and rulebook or handbook extraction pipelines.

The cumulative book-processing summaries show that this source work was substantial rather than illustrative. In the source-first cumulative summary version v5, the repository records 11 processed book families, 16,248 raw pairs, 14,740 clean pairs, and 8,412 final rows after later row selection, with strong variation by source. For example, the summary attributes 2,746 rows to the Oxford Dictionary path, 2,222 to the Cambridge Dictionary path, and 1,835 to English Grammar Today. These counts matter because they show that the dataset story is not limited to one textbook or one synthetic source; it is the result of repeated extraction and cleaning passes over multiple pedagogical resources.

7.3 Curation and Stabilization

The curator progress report presents the project as having reached a more stable baseline through five linked improvements:

1. formalization of a v2-compatible contract;
2. cleaner phrase structure and stronger fallback-note behavior;
3. stricter validation modes and more reliable schema enforcement;
4. expanded test coverage;
5. synchronization of documentation with implemented behavior.

Whether one accepts the phrase "production-ready baseline" in full or not, the report is useful because it captures the intended direction of the implementation phase: less noise, more traceability, and stronger safety around the contract itself.

7.4 Deduplication and Quality Control

A recurring theme in the project documentation is that raw volume is not enough. The system places substantial effort into:

- filtering and capping repeated targets;
- excluding low-quality or telemetry-like note text;
- preserving CEFR balance and grammar-class coverage;
- validating note suitability and trace fields;
- reducing datasets to examples that remain useful after strict normalization.

The practical effect of this policy is visible in the processed dataset statistics. One hybrid T5 contract-target dataset built from the projected corpus started with 2,510 rows and ended with only 380 rows after deduplication and balancing, with 2,130 rows skipped by target-cap controls alone. This is a strong sign that the project prioritized controllable target diversity over raw size. It also explains why some intermediate model results remained weak: the engineering decision to compress and normalize the target space trades scale for discipline.

7.5 Placeholder Construction

The note-generation path appears to depend critically on converting enriched linguistic structures into more abstract templates. This is the point where dataset engineering and the context–pattern duality meet: the abstraction strategy is not only a training trick, but part of how the project reframes the pedagogical signal.

The canonical process document dated 2026-03-21 makes that transformation explicit. It defines a ten-step path from book-derived `notation + content` pairs to inference-time rendering:

1. extract notation and paired content from source material;
2. build a contract tree from the content;
3. analyze the notation and convert it into placeholder-based template text;
4. constrain placeholders to attributes available in the paired contract;
5. train on templated notation rather than on fully rendered final notes;
6. at inference time, rebuild the contract for new content;
7. let the model output a notation template with allowed placeholders only;
8. fill those placeholders from the contract;
9. write the rendered note back into the tree.

This is a stronger and more specific claim than "the system uses placeholders." It shows that placeholders are intended to sit at the centre of the full data path from source extraction to runtime note realization.

The later slot-normalized projection artifacts make this even more concrete. In the v17 slot-normalized projection report, the repository records 3,505 projected rows, including 1,984 sentence-level slot-templated rows and 6,584 phrase-level slot-templated rows, with additional lexicalized-note repairs recorded explicitly. This demonstrates that placeholder construction was not only a conceptual strategy; it was operationalized as a measurable transformation stage over a large projected corpus.

7.6 Template Opportunity Analysis

The repository also contains reports that measure not just what was extracted, but what remained uncovered by the current template inventory. A template-opportunities report over the slot-normalized projection records 513 exact sentence families in projection, of which 205 had book notes and 202 already had book slot templates. The same report identifies phrase families with much weaker template coverage, especially around relative-clause-heavy phrase patterns. This is important because it shows that dataset engineering in ELA included gap analysis, not only corpus growth. The project did not simply build a dataset and train on it; it also measured where the current template space failed to cover the observed grammar families.

7.7 Measured Template Space

The classification-oriented template datasets show that even the template inventory itself was treated as a measurable object. One sentence-multilabel template dataset built from

the covered projection reports 1,774 rows after filtering, 48 total template identifiers, 42 sentence template types, and 6 phrase template types. These counts matter because they convert the abstract idea of "template diversity" into something inspectable. They also help explain why some model and coverage decisions in later chapters could be made with more confidence: the project had begun to quantify the shape of its pedagogical pattern space rather than relying on intuition alone.

7.8 From Free-Form T5 to Controlled Rendering

The project documentation shows a clear evolution away from unconstrained note generation. By March 2026, the preferred architecture was no longer "raw text to T5 note." The intended flow had become:

1. validated linguistic contract;
2. deterministic structural enrichment;
3. CEFR prediction and blueprint generation;
4. contract-to-template transformation;
5. controlled lexical realization from that template;
6. post-render validation that classifier-owned and contract-owned fields remain unchanged.

This evolution is important for the thesis because it supports a more mature claim than "the system uses T5." The stronger claim is that generation is being progressively confined to the narrowest part of the problem where wording variation is desirable but structural drift is unacceptable.

7.9 How the Dataset Strategy Evolved: From Raw Corpora to Grammar Books

The dataset construction strategy reached its current form through several distinct phases, each motivated by a concrete limitation discovered in the previous one. Understanding this evolution is important because the final "closed" architecture is not an initial design choice but the result of iterative problem-solving.

Phase 1: Raw Corpora and Free-Form Generation

The earliest dataset version ingested open-licence corpora—UD English Web Treebank, OANC, Tatoeba, and Wikinews—producing around 3,000 sentences and their parsed

representations. T5 was trained to output a free-form pedagogical note given a spaCy contract describing the sentence. Notes were unconstrained: the model could write anything. The variability of output appeared high, but inspection revealed that the model quickly converged on a small set of generic phrasings that were technically grammatical but offered little pedagogical value.

Phase 2: Grammar Blueprints

To stabilize output quality, the project introduced a library of over 150 *grammar blueprints*—hand-written, rule-based note templates keyed to specific grammatical classes such as `present_simple_affirmative`, `relative_clause`, or `passive_voice`. Each blueprint provided a fixed note at three CEFR tiers (elementary, intermediate, advanced). Blueprints served as a deterministic fallback when model output failed quality gates. The approach improved consistency, but the inverse problem now became clear: the blueprint library itself was too coarse. The same note appeared thousands of times in the projected corpus, and the model had no incentive to learn anything except which blueprint to select.

Phase 3: Slot-Based Templates

The next step was to parametrize blueprints with named placeholders. Instead of a fixed string, a note template could contain `{{HEAD_NOUN}}`, `{{PREPOSITION}}`, `{{OBJECT_NP}}`, or `{{RELATIVE_MARKER}}`—fields resolved at projection time from the linguistic contract of the matched instance. This made each template potentially produce a distinct note for every sentence it matched. The technical infrastructure for this—slot validation, contract-to-slot resolution, template registry—was introduced in March 2026 and is described in more detail in the Placeholder Construction section above. The placeholder system worked, but exposed a new bottleneck: the repository contained only a few dozen manually authored template variants. Corpus projection amplified those templates across many sentences, but the underlying pattern space remained narrow.

Phase 4: Grammar Books as the Source of Patterns

The diagnostic finding, documented explicitly in the template expansion strategy report for v16, was that *the main bottleneck is insufficient template granularity.* The solution was to stop authoring templates manually and instead extract them from existing pedagogical sources. Specialized grammar handbooks—Collins COBUILD English Grammar, George Yule's *Explaining English Grammar*, Betty Azar's *Basic English Grammar*, Mark Lester's *English Grammar Drills*, Farlex *Complete English Grammar Rules*, and others—contain thousands of author-written explanations, each paired with a curated example sentence. These are exactly the slot templates the project needed: validated by domain experts,

covering a wide range of constructions, and already matched to example sentences that could drive the contract extraction step. A book extraction engine was built to parse EPUB, PDF, and DJVU formats and output the same notation–content pair format used throughout the pipeline.

Canonical Training and Inference Flow

The canonical process for book-to-dataset-to-inference, documented explicitly on 2026-03-21, fixes the full flow in ten steps. The first six steps produce the training dataset; the last four describe what happens at inference time for a new sentence.

Training path (steps 1–6). A grammar book provides pairs of *notation* (the author's pedagogical explanation) and *content* (the example sentence). The content is parsed by spaCy into a linguistic contract tree. The notation is analysed and converted into a *template*: the same explanation text, but with content-specific terms replaced by named placeholders such as `{{HEAD_NOUN}}` or `{{PREPOSITION}}`. Crucially, placeholders may reference only attributes that exist in the contract built from the paired content—the contract is the sole source of allowed slot values. Each book pair then becomes one T5 training row: the input is the contract context together with the template structure; the *target is the template text with placeholders intact*, not the rendered final note.

Inference path (steps 7–10). When a new sentence arrives from a user, it passes through the same spaCy parser, producing a contract with the same schema as the training contracts. The T5 model, trained on templated targets, outputs a notation template with placeholders. Those placeholders are then filled from the very same contract that was the model's input—the contract serves double duty as both the generation context and the slot-value source. The filled template is the rendered pedagogical note, which is saved back into the contract tree under `linguistic_notes`.

This symmetry—the contract is the context at training time, and the same contract schema provides the slot values at inference time—is the technical foundation of controlled note generation. The model never sees raw unbounded text as a target and never invents structure; it only realises wording for a template derived from and constrained by the contract.

Field Ownership

The architecture formalises ownership of every contract field across three layers, as fixed in the field ownership document dated 2026-03-04:

- **spaCy and deterministic rules** own all structural fields: the tree shape, node types, part of speech, dependency labels, morphology, TAM fields (tense, aspect, mood, voice, finiteness), grammar classes, and note blueprints. These fields are set

once and cannot be modified by any downstream model.

- **Tabular ML (XGBoost)** owns `cefr_level`. This is the only field produced by a trained classifier in the production path. It is treated as a prediction, not as structural truth, and is never overwritten by the note generation step.
- **T5** owns only `linguistic_notes`—the user-facing wording. It receives the blueprint text (derived from grammar classes) and the contract context as input; it may only produce note text. It is explicitly forbidden from changing any classifier-owned or structure-owned field.

This ownership matrix is the reason the system is called *contract-safe*: the contract is not a free-form generation output but a structured record whose fields are owned by specific, accountable sources.

Three-Tier CEFR Note Design

A deliberate three-tier design governs how note templates are treated at different proficiency levels:

- **Elementary (A1–A2)**: fixed passthrough templates. Notes describe simple, unambiguous rules that do not vary with the surface content of the matched sentence. They serve as a lower-bound baseline in evaluation.
- **Intermediate (B1–B2)**: slot-templated generation. This is the primary generation task, where the same abstract pattern produces a different, grounded note for each matched sentence via contract-driven slot filling.
- **Advanced (C1–C2)**: fixed passthrough templates. Notes describe complex, construction-level rules that are likewise invariant to surface content, acting as an upper-bound comparison baseline.

By treating the boundary tiers as fixed, the project creates two natural comparison groups that bracket the range achievable by the generation system. Concrete examples from the actual training data are provided in Appendix C.

Chapter 8

Evaluation

8.1 Evaluation Approach

The evaluation of ELA must be understood as layered rather than singular. There is no one metric that validates the entire system. Instead, different subsystems require different forms of evidence:

- contract validity and structural correctness;
- CEFR prediction quality;
- pedagogical note generation quality;
- media pipeline behavior;
- end-to-end runtime usability.

8.2 Evaluation Method

The thesis uses a mixed evidence strategy rather than a single benchmark suite. Evaluation evidence is drawn from:

- structured project documents describing architectural intent and pipeline evolution;
- sample contract artifacts and schema definitions;
- explicit evaluation JSON and report checkpoints;
- broad unit-test coverage across classifier, runtime, media, contract, and validation layers;
- targeted reproduction of failing tests in the current codebase to identify active regressions.

This is appropriate for the current state of ELA because the project is simultaneously a research artifact, a software system, and a still-evolving product. A thesis that ignored any one of those layers would misrepresent the actual nature of the work.

8.3 What the Current Evidence Supports

The project contains substantial test infrastructure and multiple evaluation artifacts. This supports several defensible conclusions:

- the codebase is not a loose prototype but a systematically exercised system;
- CEFR and note-generation paths were treated as measurable modelling problems rather than hand-waved add-ons;
- media, runtime, and persistence layers were tested as first-class concerns.

8.4 Classifier Path and Controlled Generation Evidence

One useful checkpoint in the repository is the phase-1 classifier execution report dated 2026-02-25. That report confirms an orchestrated training and quality-cycle run, classifier metadata generation, runtime sentence-contract validation, and controlled note rewriting behavior. At the same time, the same report also reveals a limitation that is academically important: the baseline freeze at that point still used a very small training set. This means the existence of the pipeline is strongly supported, while any broad performance claim should remain tied to the exact dataset version that produced it.

This early checkpoint becomes much more informative when read together with later classifier artifacts. The repository contains a progression from this tiny baseline freeze toward a substantially larger full-ladder tabular run. That progression matters because it shows that the classifier path did not remain a proof of concept. It was scaled, compared, and stress-tested as the project matured.

8.5 Concrete Tabular Evaluation Snapshot

Another useful artifact is the full-ladder tabular evaluation snapshot dated 2026-03-05. Unlike broad narrative claims, this file gives a concrete checkpoint with explicit split sizes and summary metrics:

Table 8.1: Selected values from the 2026-03-05 tabular joint-profile evaluation artifact

Metric or count	Value
Training rows	8211
Development rows	1182
Test rows	1196
Joint development accuracy	0.9797
Joint test accuracy	0.9849
Grammar development macro-F1	0.8088
Grammar test macro-F1	0.8231
Runtime smoke check	pass

This snapshot is useful for two reasons. First, it proves that at least one later evaluation stage moved far beyond the tiny baseline freeze described in the February phase-1 report. Second, it shows why metric claims must be attributed carefully: different documents in the repository reflect different phases, datasets, and evaluation scopes. For the thesis, the safest strategy is to treat metrics as checkpoint-specific rather than as timeless project constants.

8.6 Comparative Evidence and Intermediate Results

The evaluation story becomes stronger when the intermediate and comparative artifacts are treated as part of the evidence rather than ignored as noise. For example, the March 2026 comparison between the tabular full-ladder path and the DeBERTa-based full-ladder path shows that the tabular route became the stronger practical choice for the production-oriented stack. In that comparison, the DeBERTa CEFR-only hygiene smoke result achieved only 0.1612 accuracy and 0.0463 macro-F1, while collapsing almost all predictions to the ‘C1‘ class. This is a valuable engineering result because it explains why the project moved away from treating the heavier classifier path as the obvious runtime default.

The note-generation artifacts tell a similarly important story. Intermediate T5 evaluation reports in directories such as `results_llm_notes_refresh_data_processed_e2` show structurally valid training and evaluation runs, but still very weak exact-match behavior on the measured target space. Earlier template-id and refresh experiments therefore matter as negative evidence: they show that simply running T5 over a notation-like target did not automatically produce useful pedagogical output. The later placeholder and controlled-rendering direction should be understood as a response to those weak intermediate results rather than as an arbitrary conceptual preference.

This layered reading of the artifacts is important for the thesis. A convincing evaluation chapter should not present only the best surviving checkpoint. It should show that the project made decisions in response to evidence: weak template-id target diversity, weak practical fallback behavior, tiny early classifier datasets, failed or underperforming comparative routes, and then stronger later checkpoints that justified a narrower architecture.

8.7 What the Current Evidence Does Not Fully Support

At the same time, the current development state also places important limits on the thesis claims. The current codebase is not fully green under the project's own unit-test suite. Targeted failures remain in:

- node-level translation propagation;
- phrase and sentence skeleton behavior;
- validation-mode consistency around TAM fields;
- ASR sentence resegmentation and metadata filtering;
- parts of the runtime-service integration boundary.

This matters because it affects how evaluation should be written. The system can be defended as substantial and technically serious, but not as fully stabilized across every path.

8.8 Representative Findings from Review

The implementation review identified the following representative engineering realities:

- the runtime layer strongly validates frontend-visible payloads, which is architecturally good, but currently exposes payload-shape mismatches;
- the skeleton builder remains a critical subsystem, and regressions there have direct pedagogical consequences because phrase structure is learner-visible;
- translation quality is not only a linguistic issue but also a structural one, because wrong projection to child nodes changes what the learner sees as local meaning;
- the media path is operationally ambitious, but multimodal support is more fragile than the text path.

8.9 Overall Assessment

Overall, the project evaluates well as a research-engineering system:

- it is broad enough to demonstrate full-stack integration;
- it is deep enough to support concrete architectural and modelling discussion;
- it is honest enough, when reviewed carefully, to show the gap between implemented ambition and production hardening;
- it preserves enough intermediate evidence to explain why later design decisions were made.

8.10 End-to-End Integration Perspective

The PWA integration documentation provides one more useful perspective on evaluation. It identifies concrete failure classes that unit tests alone may miss: browser-side model availability, service-worker cache drift, ffmpeg-related rendering failures, missing proxy headers, and user-flow stalls during long-running media processing. For the thesis, this matters because it broadens the meaning of "system validation." In ELA, validation is not only about linguistic correctness; it is also about whether a learner can move from upload to usable artifact through a stable interface path.

8.11 Objectives-to-Evidence View

Table 8.2: High-level objective-to-evidence mapping for the current thesis

Objective	Intended outcome	Current evidence type
O1	Multimodal pipeline from text/audio/video to sentence-level analysis	runtime modules, integration-test guide, API/service code, unit tests
O2	Stable hierarchical contract carrying structural and pedagogical metadata	sample contract, validator, schemas, curator report, contract-safe architecture docs
O3	Learner-facing interface for exploration, export, and workflow control	React/Vite frontend, runtime API endpoints, visualizer-related implementation artifacts and docs
O4	Improved note-generation generalization through structural abstraction	template-process docs, template-expansion strategy, T5/placeholder narrative in project docs

Chapter 9

Discussion, Limitations, and Future Work

9.1 Discussion

The implemented ELA system supports the central thesis direction: authentic language can be made more accessible through layered structural explanation rather than through simplification. The combination of hierarchical representation, deterministic truth layers, and learner-facing visualization remains the strongest and most coherent aspect of the project.

The most interesting deeper result is that abstraction appears to matter more than raw model complexity in pedagogical generation. The placeholder-based route suggests that the project achieved its clearest modelling gains not by escalating model size, but by refining representational form. In practical terms, the project moved from asking a model to memorize many surface realizations toward asking it to learn a more stable mapping between structural context and explanatory pattern.

This is also where the engineering and research layers of the thesis meet most clearly. The same design instinct that pushed the runtime toward stricter contracts and deterministic boundaries also pushed the note-generation path toward canonicalized training signals. In both cases, the project became stronger when implicit variation was converted into explicit structure.

9.2 From LLM Pattern Learning to a General Scientific Method: A Pattern-Sequence Hypothesis

The hypothesis formalised in this section did not originate as an abstract theoretical claim. It emerged through a sequence of practical observations during the ELA implementation work and was subsequently confirmed in a second, physically unrelated domain. The

fact that the same core idea surfaced independently in two domains — natural language pedagogy and meteorological forecasting — is what elevates it from an engineering note to a candidate general principle.

Origin in the ELA Pipeline Failure

The first encounter with the principle came from a practical failure rather than a design intention. When the training signal for the T5 note-generation path preserved full lexical surface detail, the model overfit to specific examples rather than learning transferable pedagogical patterns. The template-ID experiment of February 2026 made this concrete: after target-space balancing, the effective target inventory collapsed to only ten unique template identifiers, and the fallback rate on the small regression probe reached 1.0. The model had learned nothing useful because the surface form carried more variation than the structure — and the model, given no incentive to the contrary, memorised the variation rather than the underlying pattern. The corrective move — replacing raw notation with placeholder-bearing structural templates — produced qualitatively different behaviour. The model began to generalize across vocabulary while preserving grammatical interpretation. This was the first empirical encounter with the claim: the structural pattern, not the surface form, carries the transferable signal.

A Historical Precedent: Mendeleev's Grammar

The intellectual analogy that clarifies this observation most sharply predates machine learning by more than a century. Mendeleev did not discover new chemical elements. What he discovered was that elements, when arranged by their intrinsic properties, *form repeating patterns* — and that those patterns are predictive. When the sequence of known elements violated the expected structure, he did not discard the anomaly. He left an empty cell and predicted the properties of the missing element from the constraints imposed by its position in the pattern. The element was subsequently found.

The method, stripped of its chemical content, consists of three steps: identify the minimal analytical unit of description appropriate to the domain, classify it, study the grammar of its sequential arrangements, and use that grammar to forecast. The Periodic Table is not a list of elements; it is a grammar of elemental transitions. This is precisely what the ELA and weather forecasting projects each implement in their respective domains. In neither case does the model learn to predict the raw surface. In both cases, it learns to predict the *type and structure of the next analytical unit* in a sequential grammar.

Language as the First Domain

In ELA, the analytical unit is the Recursive Linguistic Element node. Each sentence, phrase, and word node is described not by its specific lexical content but by its structural type, grammatical role, tense-aspect-mood classification, and position in the hierarchy. The placeholder-based training signal then asks the model to predict which structural pattern applies to a new example — which note template, which grammatical regime — rather than to memorise which words appeared in training. This is the open phase of the duality: many contextual examples, each with different surface vocabulary, carve a stable internal pattern space. In the closed inference phase, the frozen pattern then imposes structure on whatever new context arrives. This statement is formulated explicitly in the core hypothesis:

> Effective training of an LLM is achieved when both the input and the output are represented as patterns with placeholders, and variability is realised not as a separate collection of individual cases, but as a structured system of filling those placeholders within the pattern.
>
> This gives rise to the hypothesis that, in open systems, context forms patterns, whereas in closed systems, stable patterns structure the permissible context of their realisation.
>
> In turn, this suggests that the complexity of natural systems is determined not by an infinite multitude of variants, but by a limited set of stable patterns through which those variants become possible, reproducible, and interpretable.

Weather Forecasting as the Second Domain

The most direct external evidence for the generality of this approach comes from parallel work on single-station meteorological forecasting [15] conducted at Knock Airport, Ireland, using Met Éireann historical hourly data covering the period 2020–2026. In that system, raw sensor measurements are not fed directly to a model. Instead, the time series is first segmented into analytical intervals, each described by one of a small library of ODE-derived equation types: `constant`, `linear`, `exponential`, `harmonic`, and `damped_harmonic` for temperature, and a three-type vocabulary for pressure and wind speed. The choice of equation type and its fitted parameters constitute the vocabulary of meteorological regimes in precisely the same sense that RLE node types constitute the vocabulary of grammatical regimes in ELA.

A transformer encoder is then trained to predict the *sequence* of future ODE segment descriptors from a window of past ones. The model receives no atmospheric physics, no spatial observations, and no numerical weather prediction output. It learns only the

sequential grammar of analytical meteorological units from six years of a single station's history. On a seven-day evaluation period in February 2026, the ensemble achieves a pressure mean absolute error of 0.89 hPa at the 12-hour horizon, a 34% improvement over naive persistence (1.35 hPa), using no domain physics whatsoever. This is not a claim of superiority over operational numerical weather prediction systems; it is a proof of concept that the *grammar of physical regimes is learnable from sequential analytical units alone*, exactly as the grammar of linguistic regimes proved learnable in ELA.

The structural parallel between the two domains is summarised in Table 9.1.

Table 9.1: Structural parallel between the ELA and weather forecasting pattern-sequence approaches

Concept	ELA (language domain)	Weather forecasting domain
Analytical unit	RLE node (sentence, phrase, word)	ODE segment (constant, linear, exponential, harmonic)
Unit descriptor	type, TAM, grammatical role, CEFR level	equation type, fitted parameters, duration
Sequence model	transformer over RLE node sequences	transformer over ODE segment sequences
Training signal	structural templates with placeholders	segment descriptor vectors
Predicted output	next structural pattern and its template slots	next segment type and fitted parameters
Decoded output	learner-facing pedagogical note	hourly meteorological forecast values

Formulation of the General Method

These two cases suggest a three-step method that may apply to any sufficiently regular natural or artificial process:

1. **Decompose.** Identify the minimal analytical unit of description appropriate to the domain. The unit should be expressive enough to capture the essential structure of a regime, compact enough to form a finite vocabulary, and grounded in domain knowledge rather than derived purely from statistical clustering. In language, the unit is the grammatical node with its TAM and role attributes. In meteorology, it is the ODE segment type with its fitted parameters.

2. **Classify and sequence.** Observe that naturally occurring instances of the process form sequences of these units. Classify the units, record their transitions, and treat the sequence as the primary object of study rather than the raw surface values.

The Periodic Table is the canonical example of this step applied to chemistry: once elements are classified by their properties, the grammar of their arrangement becomes visible and predictive.

3. **Learn the grammar and predict.** Apply a sequence model to the classified unit stream. Because the model operates on analytical descriptors rather than raw measurements, it learns the structural regularities — the grammar — of regime transitions. Predictions are then decoded back to surface form through the analytical units themselves, preserving interpretability at every step.

A New Methodology of Scientific Inquiry

The pattern-sequence approach is worth situating not only as a machine-learning technique but as a methodology of scientific inquiry in its own right — one that became practically realisable only with the arrival of modern sequence models. Prior to the transformer era, identifying the analytical units of a domain and studying their sequential grammar was work reserved for domain experts investing years of manual analysis. The Periodic Table took Mendeleev a lifetime; the grammatical rules of natural language occupied generations of linguists; the ODE taxonomy of meteorological regimes exists only as informal folklore among forecasters. What the AI revolution makes possible is the *automation of the grammar-learning step*: once a domain can be expressed as a sequence of interpretable analytical units, a transformer can extract the sequential regularities from historical data at a scale no human analyst could match.

This positions the three-step method — decompose, classify, learn the grammar — as a candidate general framework for data-driven scientific understanding, distinct from both classical hypothesis-driven science and from the black-box statistical learning that characterises much of modern deep learning. It is interpretable because the units are analytically defined; it is powerful because the grammar is learned from data; and it is generalisable because the same architectural pattern — segment, describe, sequence-model — applies wherever recurring analytical regimes can be identified. The two domains examined in this thesis, natural language and meteorology, are early instances of what may prove to be a broader shift in how structured knowledge is extracted from complex systems.

Scope and Limitations of the Hypothesis

The hypothesis should be understood as an empirical claim derived from two implemented cases, not as a formal theorem. Several important qualifications apply.

First, the quality of the analytical decomposition is a decisive factor. If the vocabulary of units is too coarse, important distinctions collapse; if it is too fine, the model faces a large and sparsely populated unit space that is difficult to learn from. In both ELA

and the weather project, vocabulary size was determined through domain knowledge and iterative refinement rather than by automated search, and that design investment is not free.

Second, the generalization gains observed in both domains are bounded by evaluation scope. ELA's placeholder-based note generation was evaluated on a controlled annotation set; the weather forecasting result covers one week at one station during a challenging frontal passage. Neither result is sufficient to claim general superiority over all alternative approaches.

Third, the claim is not that all complexity reduces to patterns in a mystical sense. The hypothesis is more practical: in domains where analytical decomposition is feasible, learning the grammar of analytical unit sequences is a more efficient and more interpretable strategy than learning directly from raw surface data. Whether that efficiency advantage survives in richer, noisier, or higher-dimensional domains is an open research question.

The three-step method, the Mendeleev analogy, and the context–pattern duality together constitute the research contribution of this thesis that extends beyond ELA's engineering work. They represent a design philosophy tested in two domains and presented here as a candidate framework for further systematic investigation.

9.3 Limitations

The current limitations are both scientific and engineering-related:

- some reported metrics still require careful reconciliation with the precise artifacts that produced them;
- the current codebase contains unresolved regressions in the core pipeline;
- multimodal support is more fragile than text-only processing;
- some claims inherited from earlier project narratives need weaker wording in light of the current implementation state;
- the project still lacks a full user-study layer capable of demonstrating educational impact beyond technical plausibility.

The repository planning artifacts show that these limits are not abstract caveats added after the fact. Several unfinished items in the working TODO trail correspond directly to the current discussion. Examples include moving `grammar_classes` ownership fully into the rules-first runtime path, deciding whether DeBERTa remains a research-only path or is fully retired from the mainline runtime story, and tightening provenance and licensing evidence for some advanced-corpus packages. This matters because the remaining work

is now relatively concrete: the project is no longer blocked by lack of direction, but by stabilization, ownership cleanup, and proof discipline.

Table 9.2: Current implementation limitations and their practical impact

Limitation area	Practical impact on the thesis or system
Metric fragmentation across phases	prevents careless use of a single headline number for the whole project
Runtime-service payload mismatch	shows that architectural strictness currently outpaces some frontend/runtime integration assumptions
Skeleton and translation regressions	directly affect learner-visible tree quality and node-level interpretation
Multimodal fragility	makes text processing more stable than audio/video processing in the current codebase
Lack of large-scale user study	limits pedagogical claims to technical plausibility and architecture-backed rationale

9.4 Future Work

The most immediate future work falls into three clusters:

1. **Stabilization.** Resolve current regressions in skeleton construction, translation projection, runtime validation boundaries, and media sentence preparation.

2. **Model path maturation.** Continue the abstraction-based note-generation line, expand dataset evidence, and strengthen the empirical basis of the context–pattern duality claim.

3. **Product hardening.** Improve media playback, subtitle synchronization, packaging, deployment consistency, and frontend/runtime ergonomics.

The recent repository evidence suggests that this future work is likely to remain iterative rather than monolithic. Many 2026 implementation passes are not feature additions but corrective passes over translation routing, batch behavior, provider selection, stale translation cleanup, media caching, and rendering reliability. That pattern implies that the next mature phase of ELA will depend less on adding yet another model and more on reducing boundary mistakes between already existing components.

Beyond those immediate priorities, the documentation also points toward longer-horizon directions: tighter offline packaging of runtime models, richer translation quality gates, broader corpus growth, stronger observability around pipeline failures, more disciplined separation between classification, controlled rendering, and user-facing editing

workflows, and eventually a user-study layer that can test whether the explainable interface actually improves learner comprehension and retention under realistic use.

Chapter 10

Conclusion

This thesis has presented ELA as the continuation of an earlier research programme into explainable support for authentic English input. The project has evolved from a concept and prototype direction into a substantial full-stack implementation built around the RLE contract, a rules-first linguistic pipeline, multimodal processing, and a learner-facing frontend.

The core technical lesson of the work is twofold. First, explainability requires architecture, not just model output: contracts, validators, runtime boundaries, and structural representations matter. Second, abstraction in the training signal appears to be a decisive factor in pedagogical note generation, motivating the context–pattern duality as the main interpretive research contribution of this phase, within the limits of the current evidence base.

The project is not finished. Its current codebase still exposes unresolved regressions and integration mismatches, especially in the runtime and multimodal paths. Yet that incompleteness does not weaken the value of the thesis; it clarifies it. ELA is best understood not as a polished consumer product, but as a serious research-engineering system whose value lies in what has already been made explicit: a reusable architecture, a contract-centered representation model, a credible multimodal workflow, and a modelling direction that can be tested further rather than merely asserted.

In that sense, the continuation from the earlier research thesis is successful. The original pedagogical vision was not abandoned during implementation; it was forced into concrete interfaces, validators, services, datasets, and failure cases. That transition from concept to accountable system is the real achievement documented in this report.

Chapter A

Repository Evidence Sources

Table A.1 summarizes the main repository artifacts used to support the thesis claims. The purpose of this appendix is not to reproduce those documents in full, but to make explicit which internal sources informed architectural, evaluative, and historical statements in the main body.

Table A.1: Representative repository evidence sources used in this thesis

Repository artifact	**Thesis role**
Project README set	high-level system narrative, feature inventory, and implementation scope
Full pipeline documentation	broad architectural description of the pipeline and runtime responsibilities
Technical specification dated 2026-03-20	system-level technical specification and contract-oriented implementation detail
Template expansion strategy v16	evidence for controlled rendering and placeholder-driven note-generation strategy
Canonical book-to-dataset process note dated 2026-03-21	evidence for dataset construction and contract-to-template processing flow
Phase-1 classifier execution report dated 2026-02-25	checkpoint evidence for classifier orchestration and small-baseline limitations
Full-ladder tabular evaluation artifact dated 2026-03-05	concrete metrics snapshot for later-stage tabular evaluation
PWA integration test note	evidence for end-to-end PWA and runtime interaction testing
Docker deployment note	deployment and packaging evidence for the implementation story
Current unit-test suite and targeted failing tests	evidence for both maturity and unresolved regressions in the present codebase

Chapter B

Evaluation Artifact Index

The project contains multiple evaluation checkpoints rather than a single immutable benchmark. Table B.1 indexes the main classes of evidence used in the report and clarifies what each can and cannot support academically.

Table B.1: Evidence categories and their interpretive strength in the current thesis

Evidence class	**Strength level**	Appropriate use in the thesis
Running code and compiled thesis output	**High**	supports claims that the project exists as an operational software artifact with reproducible document output
Contract schema, validators, and runtime interfaces	**High**	supports claims about architectural intent, field ownership, and explainability-oriented system design
Evaluation JSON checkpoints and classifier reports	**Medium to high**	supports checkpoint-specific metric claims when the producing dataset and phase are stated explicitly
Integration tests and runtime smoke paths	**Medium**	supports claims that cross-layer workflows were implemented and exercised end to end
Targeted failing tests in the current codebase	**High for limitations**	supports current-state limitation claims and prevents overstating maturity
Earlier thesis and narrative planning documents	**Medium**	supports continuity and design intent, but not unconditional statements about the current codebase

Chapter C

Pipeline Template Examples

This appendix provides concrete examples drawn from the actual training data generated by the closed pipeline described in Section 7.10. As explained there, the pipeline uses a three-tier design:

- **Elementary (A1–A2)**: fixed passthrough templates, used as a lower-bound comparison baseline;
- **Intermediate (B1–B2)**: slot-templated generation, the primary modelling target;
- **Advanced (C1–C2)**: fixed passthrough templates, used as an upper-bound comparison baseline.

For each example the table shows the source phrase or sentence, the grammatical topic as labeled in the source book, the note template (with any `{{SLOT}}` placeholders shown explicitly for intermediate examples), and the final rendered note that would be displayed to the learner.

Elementary Level (A1–A2): Passthrough Notes

At the elementary tier the note text is typically short, unambiguous, and directly applicable regardless of the specific sentence. These notes are stored as passthrough templates with no slots.

Table C.1: Elementary-level passthrough note examples from the training corpus

Source phrase / sentence	**Topic**	**Note (passthrough template)**
I'm going to go downtown tomorrow.	be going to future	Be going to is used to talk about a future plan or intention.
Ann isn't going to study tonight.	be going to negative	In negative be going to clauses, not follows be.
Are you going to come to class tomorrow?	be going to yes-no question	A yes-no question with be going to is formed by placing be before the subject.
What time are you going to eat dinner tonight?	be going to wh-question	A wh-question with be going to begins with the wh-expression and keeps be before the subject.
There is a book on my desk.	existential there basic	There is introduces the existence of a singular thing in a place or situation.

Intermediate Level (B1–B2): Slot-Templated Notes

At the intermediate tier many note patterns contain references to specific grammatical elements of the target phrase. These are stored as slot templates; at projection time the slots are filled from the linguistic contract of the matched corpus instance, producing a grounded note for each new context.

Table C.2: Intermediate-level slot-templated note examples from the training corpus

Target phrase	**Topic**	**Note template (with slots)**	**Rendered note (slots filled)**
that you hit	relative clause antecedent	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "car".
that I met	object relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "woman".
that you talked to	stranded preposition relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "person".
for my birthday	prepositional phrase	Here, "{{OBJECT_NP}}" functions as the object of the preposition "{{PREPOSITION}}".	Here, "my birthday" functions as the object of the preposition "for".
with a hammer	prepositional phrase	This prepositional phrase contains the preposition "{{PREPOSITION}}" and the object "{{OBJECT_NP}}".	This prepositional phrase contains the preposition "with" and the object "a hammer".
who I was talking about	relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "woman".

The slot mechanism makes the difference across the three tiers concrete. Elementary and advanced notes are self-contained pedagogical rules whose text does not change with the sentence: they are passthrough templates. Intermediate notes are abstract patterns

that become specific only when instantiated against a real contract: they are slotted templates. All three forms pass through the same closed pipeline; only the template kind—passthrough versus slotted—differs. The two fixed tiers serve as natural baselines that bracket the quality range achievable by the generation system.

Bibliography

[1] A. Gilmore, "Authentic materials and authenticity in foreign language learning," *Language Teaching*, vol. 40, no. 2, pp. 97–118, 2007.

[2] W. Guariento and J. Morley, "Text and task authenticity in the EFL classroom," *ELT Journal*, vol. 55, no. 4, pp. 347–353, 2001.

[3] S. A. Berardo, "The use of authentic materials in the teaching of reading," *The Reading Matrix*, vol. 6, no. 2, pp. 60–69, 2006.

[4] G. Levy, *Computer-Assisted Language Learning: Context and Conceptualization*. Oxford, UK: Clarendon Press, 1997.

[5] C. A. Chapelle, *Computer Applications in Second Language Acquisition*. Cambridge, UK: Cambridge University Press, 2001.

[6] M. Honnibal and I. Montani, "spaCy 3: Industrial-strength Natural Language Processing in Python," 2023.

[7] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[8] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *Proceedings of ICML*, 2023.

[9] A. Fan et al., "Beyond English-centric multilingual machine translation," *Journal of Machine Learning Research*, vol. 22, no. 107, pp. 1–48, 2021.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of KDD*, 2016, pp. 785–794.

[11] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[12] R. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, UK: Cambridge University Press, 2009.

[13] A. Collins, J. S. Brown, and S. E. Newman, "Cognitive apprenticeship: Teaching the crafts of reading, writing, and mathematics," in *Knowing, Learning, and Instruction*, L. B. Resnick, Ed. Hillsdale, NJ: Lawrence Erlbaum, 1989, pp. 453–494.

[14] V. Rastvorov, *ELA: English Language Assistant*. Research thesis, Munster Technological University Cork, 2025.

[15] N. Ahmad and V. Rastvorov, *ODE-Segment Sequence Forecasting: Learning Weather Pattern Grammar from Single-Station Observations*. Technical report, Munster Technological University Cork, 2026. Available: https://github.com/welsol21/weather_forecast

[16] V. Rastvorov, *ELA: English Language Assistant — Source Repository*. 2026. Available: https://github.com/welsol21/FYP_LLM